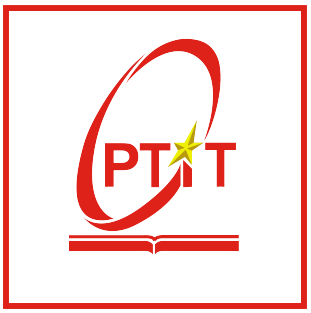
POSTS AND TELECOMMUNICATIONS
INSTITUE OF TECHNOLOGY
INFORMATION TECHNOLOGY



INTERNSHIP

REPORT

WoxionChat

Name: NGUYEN PHAM TRUNG HIEU
Student ID: B23DCCE031
Class ID: B23CQCE04-B
Instructor: Dr. KIM NGOC BACH

Hanoi

Table of contents

1 Introduction

1.1 Background and Rationale

In the era of Industry 4.0, enterprises generate and manage an enormous volume of unstructured data, including contracts, standard operating procedures (SOPs), technical manuals, reports, and internal documentation. Traditional keyword-based search systems often fail to capture semantic relationships and contextual information, making knowledge retrieval inefficient and time-consuming [1].
Recent advances in Large Language Models (LLMs) have enabled more natural and intelligent interaction with enterprise knowledge [1]. However, relying on proprietary cloud-based AI services raises significant concerns regarding data privacy, security, and operational costs. Sensitive corporate information may be transmitted to third-party servers, reducing organizational control over confidential data and creating long-term dependency on external providers.
To address these challenges, this project proposes **WoxionChat**, an on-premise enterprise AI assistant designed for secure and intelligent interaction with enterprise knowledge bases. Unlike conventional RAG systems [2] that simply retrieve relevant documents and inject them into prompts, WoxionChat adopts an **Agentic Retrieval-Augmented Generation (Agentic RAG)** architecture, where autonomous agents collaborate to plan retrieval, reason over multiple evidence sources, and generate reliable responses.
Furthermore, the system incorporates an enhanced version of **Agentic Context Engineering (ACE)** [3] to continuously organize, refine, and evolve contextual knowledge accumulated from enterprise documents and user interactions. This adaptive context management enables the model to preserve important domain-specific information and improve reasoning quality over time while maintaining data privacy through private deployment.

1.2 Research Objectives

The objectives of this research are as follows:

- To design and implement WoxionChat based on an Agentic RAG architecture for intelligent retrieval and reasoning over enterprise knowledge.
- To improve the ACE framework by introducing more effective context management and refinement mechanisms, enabling higher-quality contextual reasoning in long-running interactions.
- To fine-tune and deploy an open-source Vietnamese large language model capable of operating entirely within private infrastructure.
- To integrate multimodal document processing for various enterprise file formats while ensuring data privacy and knowledge consistency.

1.3 Research Contributions and Practical Significance

This work extends beyond building an enterprise chatbot by proposing an integrated knowledge intelligence platform. The primary contributions include:

- The development of **WoxionChat**, an on-premise Agentic RAG system capable of securely processing and reasoning over enterprise knowledge from heterogeneous document sources.

- The enhancement of the **ACE (Agentic Context Engineering)** framework through improved context evolution and management strategies, resulting in significantly stronger reasoning performance on complex tasks.
- The result of finetuning LLM in the final test at VMLU [4] benchmark is **61.95%**
- Experimental results demonstrate that the proposed improvements increase the final test accuracy from **70%** to **77%** on challenging benchmarks, indicating the effectiveness of the enhanced context engineering approach.
- The resulting platform functions as a centralized organizational knowledge assistant, supporting document analysis, employee training, policy consultation, and enterprise decision-making while preserving full control over sensitive data.

2 Theoretical framework

2.1 Retrieval-Augmented Generation (RAG)

RAG is a sophisticated technique that connects language models with reliable external data sources [2]. The process operates based on three main steps:

1. **Indexing:** Text is converted into vector embeddings and stored in a Vector Database.
2. **Retrieval:** Upon a query, the system searches for text segments with the highest mathematical similarity in a multi-dimensional space.
3. **Generation:** The LLM utilizes the context from retrieved segments to generate accurate responses, mitigating the "hallucination" phenomenon common in LLMs.

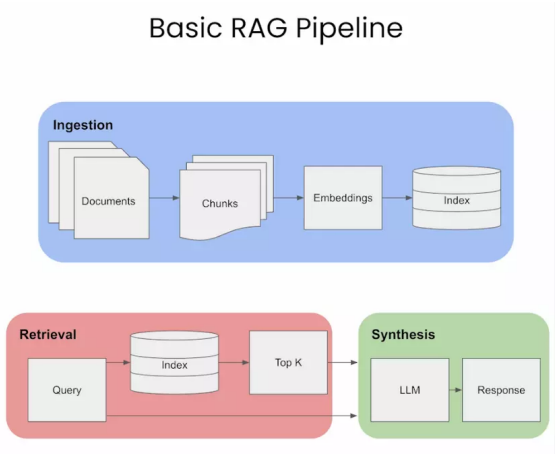


Figure 1: RAG architecture

2.2 Agentic Retrieval-Augmented Generation (Agentic RAG)

Retrieval-Augmented Generation (RAG) [2] enhances Large Language Models (LLMs) by retrieving relevant external knowledge and incorporating it into the generation process. In a

conventional RAG pipeline, a user query is encoded into a vector representation, relevant documents are retrieved from a knowledge base, and the retrieved passages are appended to the prompt for the language model to generate the final response. While this architecture effectively mitigates hallucinations and provides access to up-to-date information, it follows a relatively static retrieval strategy and lacks the ability to adapt dynamically to complex reasoning tasks.
Agentic Retrieval-Augmented Generation (Agentic RAG) extends the traditional RAG paradigm by integrating autonomous AI agents capable of planning, reasoning, and tool utilization. Instead of performing a single retrieval step, the agents can iteratively decompose complex queries, select appropriate retrieval strategies, invoke external tools or databases, evaluate intermediate results, and refine subsequent actions based on newly acquired information. This enables the system to actively construct an evidence-driven reasoning process rather than relying solely on one-pass retrieval.
Compared with conventional RAG, Agentic RAG offers several key advantages. First, it supports adaptive retrieval by dynamically selecting information sources according to the task and context. Second, it enables multi-step reasoning through iterative planning and execution, making it suitable for complex analytical queries that require evidence from multiple documents or heterogeneous data sources. Third, autonomous tool usage allows the system to interact with external services, structured databases, or enterprise APIs when necessary, thereby extending its capabilities beyond static document retrieval.

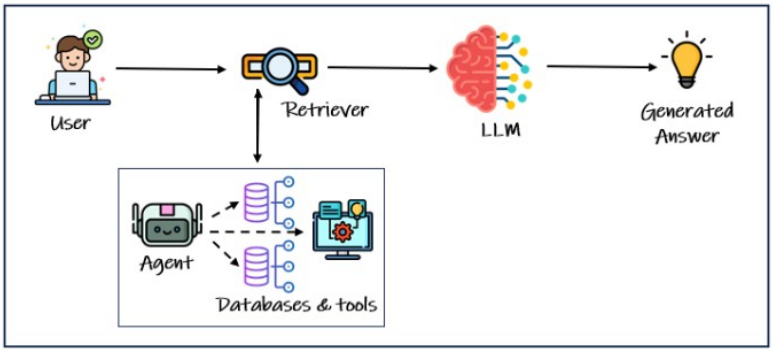


Figure 2: Simple agentic RAG architecture

2.3 Low-Rank Adaptation (LoRA)

Low-Rank Adaptation (LoRA) [5] is a parameter-efficient fine-tuning (PEFT) technique designed to reduce the computational and memory overhead associated with adapting large language models. Instead of updating all model parameters during training, LoRA freezes the original pretrained weights and introduces a pair of trainable low-rank matrices into selected layers, typically the attention projections.
Formally, let the original weight matrix be denoted as $W_0 \in \mathbb{R}^{d\times k}$. Rather than directly optimizing W_0, LoRA models the weight update as the product of two low-rank matrices:

$$\Delta W = BA,$$

where $A \in \mathbb{R}^{r\times k}$, $B \in \mathbb{R}^{d\times r}$, and $r \ll \min(d, k)$ is the rank hyperparameter. Consequently, the adapted weight matrix becomes

$$W = W_0 + \Delta W = W_0 + BA.$$

This decomposition significantly reduces the number of trainable parameters from $d \times k$ to $r \times (d + k)$, resulting in substantial savings in GPU memory consumption and training time. As a result, LoRA enables efficient fine-tuning of large-scale models on resource-constrained hardware while maintaining competitive downstream performance.
Another important advantage of LoRA is that the pretrained weights W_0 remain unchanged throughout the adaptation process. By preserving the original knowledge encoded in the base model and learning only lightweight task-specific updates, LoRA helps mitigate catastrophic forgetting while allowing the model to adapt effectively to new domains or specialized tasks.

2.4 Context Engineering

Context engineering is an emerging paradigm for improving the effectiveness of Large Language Model (LLM) based systems by optimizing the information presented to the model during inference [6]. Unlike traditional prompt engineering, which primarily focuses on crafting instructions, context engineering considers the broader problem of selecting, organizing, and maintaining the most relevant contextual information including system prompts, retrieved documents, tool outputs, conversation history, and external knowledge to maximize task performance.

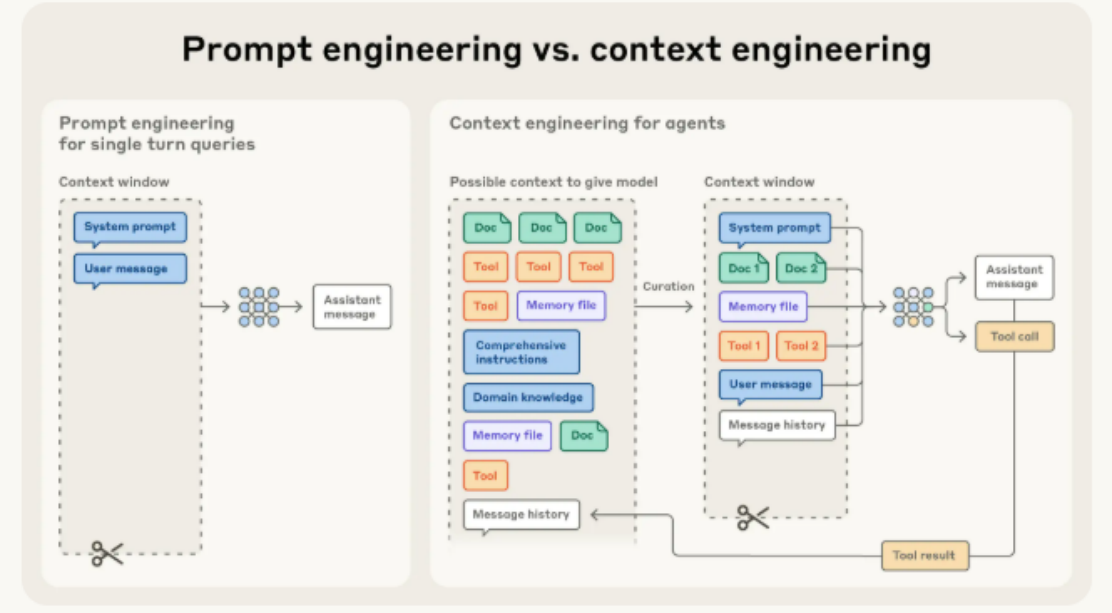


Figure 3: Context engineering

Context engineering is motivated by the limited attention capacity of transformer models. Although modern LLMs support long context windows, excessive or irrelevant information can dilute attention and degrade reasoning quality. Therefore, the objective is to curate a compact, high-signal representation that contains only the information necessary for the current task.
In enterprise AI applications, effective context engineering is particularly important because relevant knowledge is often distributed across heterogeneous sources such as internal documents, databases, APIs, and previous user interactions. A well-designed context management strategy enables the system to dynamically retrieve, prioritize, summarize, and refine information before passing it to the language model, thereby improving both response accuracy and computational

efficiency.

3 Methodology

3.1 Large Language Model Fine-Tuning

3.1.1 Dataset Construction and Filtering

A high-quality instruction dataset is essential for effective supervised fine-tuning of large language models. In this work, a multi-domain dataset was constructed by collecting question-answer pairs from diverse public resources, including websites, digital documents in PDF format, educational materials, and open-source datasets available on the Hugging Face platform. The collected data spans a broad range of domains, such as mathematics, literature, history, law, culture, science, and general knowledge, ensuring that the resulting model possesses comprehensive reasoning and knowledge capabilities across different application scenarios.
After preprocessing, deduplication, and format normalization, the initial corpus contained more than 15,000 instruction samples. However, training on all collected data indiscriminately may introduce many trivial examples that contribute little to improving reasoning performance. Therefore, an automatic filtering strategy was designed to identify samples with higher learning value. Specifically, the **Qwen3-8B** [7] was employed to evaluate each question through three independent inference attempts. The generated answers were compared with the corresponding ground-truth labels. A sample was retained in the training set if either of the following conditions was satisfied:

- The model failed to produce the correct answer after three attempts.
- The model generated inconsistent responses across the three attempts, indicating unstable reasoning or uncertainty.

This filtering process effectively selected examples that were more challenging for the language model and therefore more informative for fine-tuning. After filtering, the dataset was reduced from over 15,000 samples to approximately 6,000 high-quality instruction examples suitable for reasoning-oriented training.

3.1.2 Long Chain-of-Thought Generation and Fine-Tuning

Following dataset filtering, an additional reasoning generation stage was introduced to transform the selected instruction data into a format suitable for reasoning-centric supervised fine-tuning. Rather than training the model using only input-output pairs, each selected sample was augmented with an explicit reasoning process, producing an instruction–reasoning–answer triplet.
The primary objective of generating Long Chain-of-Thought (Long CoT) annotations is to expose the intermediate logical steps leading to the final answer. Such supervision encourages the model to learn structured problem-solving strategies instead of merely memorizing output patterns. This is particularly beneficial for tasks requiring multi-step deduction, evidence synthesis, or domain-specific analysis.
The generated reasoning traces provide detailed explanations that bridge the gap between the user query and the ground-truth answer, allowing the model to internalize complex reasoning behaviors during fine-tuning. Consequently, the resulting model is better equipped to perform systematic analysis, maintain logical consistency, and solve difficult problems that require sequential inference.
The final supervised fine-tuning dataset therefore consists of three components:

- **Instruction**: the original user question or task.
- **Reasoning**: a detailed long-form Chain-of-Thought describing the intermediate reasoning process.
- **Answer**: the final ground-truth response.

3.1.3 Training Configuration

The selected instruction–reasoning–answer dataset was used to fine-tune the **Qwen3-8B** base model using the Swift training framework with the Low-Rank Adaptation (LoRA) method. LoRA was adopted to achieve parameter-efficient fine-tuning while significantly reducing GPU memory consumption and computational cost compared with full-parameter optimization.
The model was trained for three epochs with a per-device batch size of 8 and a gradient accumulation step of 2, resulting in an effective batch size of 16. The Adam optimizer was configured with a learning rate of 5×10^{-5} and a warm-up ratio of 5% to stabilize the early stages of optimization.
To improve memory efficiency and training throughput, training used `bfloat16`, **Flash Attention 2**, and gradient checkpointing. Sequence packing reduced padding overhead by concatenating short samples into longer sequences. The maximum input length was set to 3,000 tokens to accommodate long Chain-of-Thought traces.
For parameter-efficient adaptation, LoRA modules were applied to all linear layers of the transformer architecture using a rank of 16 and a scaling factor α of 32. During training, only the LoRA parameters were updated while the original pretrained weights remained frozen, preserving the general capabilities of the base model while learning task-specific reasoning behaviors.

Table 1: Fine-tuning configuration for the proposed model.

Parameter	**Value**
Base model	Qwen3-8B
Training method	LoRA (PEFT)
LoRA rank (r)	16
LoRA alpha (α)	32
Target modules	All linear layers
Epochs	3
Per-device batch size	8
Gradient accumulation	2
Effective batch size	16
Learning rate	5×10^{-5}
Warm-up ratio	0.05
Maximum sequence length	3000 tokens
Precision	bfloat16
Attention implementation	FlashAttention-2
Gradient checkpointing	Enabled
Sequence packing	Enabled
Checkpoint interval	Every 50 steps

3.2 Agentic Context Engineering (ACE)

Despite context engineering has a lot of advantages, existing context adaptation approaches suffer from two fundamental limitations: **brevity bias** and **context collapse** [3]. **Brevity**

bias refers to the tendency of language models or optimization procedures to favor increasingly shorter prompts or summaries. During iterative refinement, rich domain-specific instructions are often compressed into concise but overly generic descriptions, resulting in the loss of critical details, specialized heuristics, and reusable reasoning patterns. Although shorter contexts reduce computational cost, they may significantly degrade performance on complex tasks that require nuanced domain knowledge.
A related issue is **context collapse**, which occurs when the context is repeatedly rewritten or summarized over multiple iterations. Instead of preserving accumulated knowledge, each successive rewrite tends to discard information that appears locally unimportant, causing irreversible degradation of the overall context. As the adaptation process continues, the context may shrink dramatically while simultaneously losing valuable reasoning strategies and historical experiences, ultimately reducing downstream task performance.

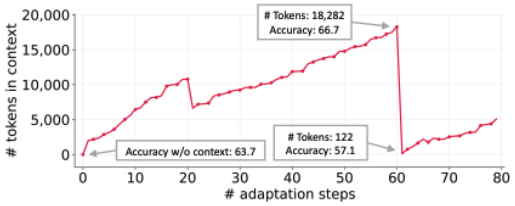


Figure 4: Context Collapse

Agentic Context Engineering (ACE) was proposed to overcome these challenges by treating context as an evolving and structured knowledge repository rather than a monolithic prompt. Instead of repeatedly rewriting the entire context, ACE represents knowledge as fine-grained playbook entries that can be incrementally updated through localized modifications. New insights are appended as delta updates, existing entries are selectively refined, and redundant information is removed through a grow-and-refine mechanism. This design preserves previously acquired knowledge while allowing the context to evolve continuously, thereby mitigating both brevity bias and context collapse [3].

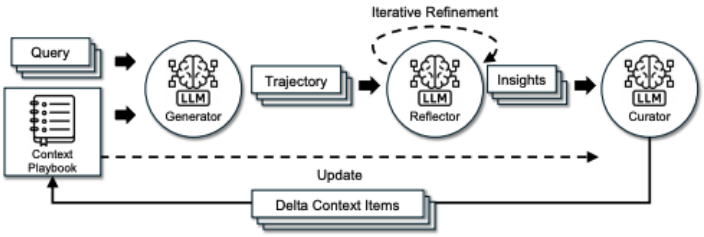


Figure 5: ACE framework

Although ACE effectively addresses the challenges of brevity bias and context collapse through incremental updates and structured playbook management, it still exhibits several limitations in practical deployment. In particular, the quality of the evolving playbook heavily depends on the **Reflector**, whose responsibility is to extract meaningful insights from execution trajectories and convert them into reusable contextual knowledge.

If the Reflector generates inaccurate analyses or hallucinates during the insight extraction process, these erroneous observations may be incorporated into the playbook by the Curator. Since future inferences rely on the accumulated playbook as contextual guidance, such errors can propagate across subsequent interactions and gradually degrade the quality of the stored knowledge. Consequently, a single incorrect reflection may trigger a cascading effect in which flawed context influences future reasoning, leading to increasingly unreliable playbooks over time.
Furthermore, as the playbook continuously expands, injecting the entire collection of accumulated experiences into the model context becomes computationally inefficient and introduces substantial redundancy. Many historical entries are only weakly related to the current query and therefore consume valuable context window capacity without providing meaningful benefits.

3.2.1 Retrieval-Augmented Execution (RAE)

The first enhancement is a **Retrieval-Augmented Execution (RAE)** mechanism. Instead of injecting the entire playbook into the model context, every playbook entry is encoded into a dense vector representation and indexed in a vector database. When processing a new query, semantic similarity search is performed to retrieve only the top-(K) playbook entries that are most relevant to the current task.
This retrieval-based strategy significantly reduces context length and computational overhead while preserving the most informative experiences. As the playbook continues to evolve over time, RAE enables scalable context construction by ensuring that only task-relevant knowledge is presented to the language model. Consequently, the model is less likely to be distracted by unrelated historical information and can allocate its attention more effectively to useful guidance.

3.2.2 Failure Memory Bank

The second enhancement introduces a dedicated **Failure Memory Bank** to explicitly capture unsuccessful reasoning trajectories. Whenever the system produces an incorrect answer or exhibits suboptimal behavior, the corresponding interaction is analyzed by the Reflector, which extracts the underlying error pattern and generates corrective insights. These failure cases are then embedded and stored in a separate memory repository.
During subsequent inference, semantic retrieval is performed over the Failure Memory Bank to identify the top-(K) historical failures that are most similar to the current query. The retrieved failure examples are provided to the Reflector as auxiliary context, allowing it to recognize analogous situations and proactively avoid repeating previously observed mistakes.

4 Analysis and Design System

4.1 Analysis

WoxionChat is designed as an enterprise AI assistant that enables intelligent interaction with internal organizational knowledge while ensuring data privacy and efficient information retrieval. The system supports the complete lifecycle of enterprise document processing, including document upload, OCR extraction, semantic chunking, vector indexing, and intelligent question answering through an Agentic RAG architecture.
From a user perspective, the system provides natural language querying over both personal and enterprise knowledge bases, enhanced by vector retrieval, reranking, and hierarchical memory management to deliver accurate and context-aware responses. In addition, the integrated Agentic

Context Engineering (ACE) framework continuously improves system performance by learning from user feedback and previous interactions.
To ensure secure operation, WoxionChat employs a role-based access control mechanism consisting of two primary roles:

- Administrator (Admin): Responsible for managing users, maintaining the enterprise knowledge base, uploading and processing shared documents, configuring system resources, and overseeing overall platform operation.
- User: Responsible for uploading personal documents, interacting with the AI assistant, querying enterprise knowledge, managing personal conversations and notes, and providing feedback to improve the system through the ACE framework.

By separating administrative and end-user privileges, the system ensures secure access to sensitive resources while providing a scalable and collaborative knowledge management platform for enterprise environments.

4.1.1 Document Upload and Management Module

The Document Upload and Management module allows users to add enterprise documents to the system so that they can be used as knowledge sources for future interactions. Users can upload various types of files, such as PDF documents, Word files, text files, and images containing textual information.
After a document is uploaded, the system processes its content and prepares it for intelligent retrieval. This ensures that the information contained in the document can later be searched and referenced when answering user queries. The module also enables users to view and manage their uploaded documents, providing a convenient way to maintain their personal knowledge resources while supporting the shared enterprise knowledge base.

4.1.2 Chatbot Interaction Module

The Chatbot Interaction module provides the primary interface through which users communicate with WoxionChat using natural language. Users can ask questions related to enterprise documents, request explanations, or seek assistance with various work-related tasks.
When a query is submitted, the system analyzes the user's request, identifies relevant information from the available knowledge sources, and generates an appropriate response. In addition to retrieving information from uploaded documents, the chatbot maintains conversational context through memory mechanisms, allowing it to understand follow-up questions and provide more coherent interactions across multiple exchanges.
The module is designed to deliver fast, accurate, and context-aware responses, enabling users to efficiently access organizational knowledge and improve daily productivity.

4.1.3 Self-Improving Module

The Self-Improving module enables WoxionChat to continuously enhance its performance by learning from previous interactions and user feedback. Instead of relying solely on the fixed knowledge of the underlying language model, the system incrementally accumulates practical experience and uses it to improve future responses.
When users identify an incorrect or incomplete answer, they can provide the expected response as feedback. The system analyzes this information to determine the cause of the error and

extracts useful lessons from the interaction. These lessons are then incorporated into an evolving knowledge repository, allowing the chatbot to refine its reasoning strategies over time.
To ensure that the accumulated knowledge remains relevant and efficient, WoxionChat retrieves only the most useful experiences related to the current query rather than using the entire history. In addition, historical failure cases are stored separately and consulted during future interactions so that the system can recognize similar situations and avoid repeating previous mistakes.
Through this continuous learning process, the self-improving module enhances the accuracy, consistency, and adaptability of WoxionChat without requiring frequent retraining of the underlying language model. This capability is particularly valuable in enterprise environments, where new knowledge and user requirements evolve over time.

4.2 Design

4.2.1 Overall System Architecture

WoxionChat adopts a modular client-server architecture that separates conventional web services from computationally intensive AI components. The system consists of three major layers:

- The presentation layer provides a web-based interface through which users can upload documents, manage personal knowledge, and interact with the AI assistant.
- The application layer is responsible for user authentication, document management, request routing, and communication with the AI engine. To improve scalability and maintainability, the Agentic RAG service is deployed as an independent service that processes retrieval and reasoning tasks separately from the web application.
- The data layer stores user accounts, uploaded documents, vector representations, conversation memories, and self-improving knowledge bases. By decoupling these components, the architecture enables efficient resource utilization while supporting future extensions and independent deployment of AI services.

4.2.2 Technology Stack

Table 2: Technology stack used in WoxionChat

Layer	Technology	Purpose
Frontend	HTML5 [8], CSS3 [9], JavaScript	Provides the web-based user interface and supports interactive user operations.
Backend	Python, Django [10]	Handles user authentication, request routing, session management, document management, and communication with AI services.
AI Orchestration	LangGraph, LangChain [11]	Implements the Agentic RAG workflow, orchestrates retrieval pipelines, and manages reasoning processes.
Large Language Model	Fine-tuned Qwen3-8B [7]	Generates context-aware responses and performs complex reasoning over retrieved information.
Embedding Model	BGE-M3 [12]	Converts documents and user queries into dense vector representations for semantic retrieval.
OCR Engine	Tesseract OCR [13]	Extracts textual information from scanned documents and image files.
Vector Database	MongoDB Atlas Vector Search [14]	Stores vector embeddings and supports semantic retrieval using vector similarity
Primary Database	MongoDB [14]	Stores user accounts, documents, semantic chunks, notes, playbooks, and failure memories.
File Storage	MongoDB GridFS [14]	Stores large uploaded files such as PDF documents and images.
Cache and Memory	Redis [15]	Maintains short-term conversation memory and improves retrieval efficiency.
Authentication	Google OAuth2, Django Authentication	Provides secure user authentication and account management.

4.2.3 Document Upload and Management Module

Uploaded documents undergo a preprocessing pipeline before becoming searchable knowledge sources. Depending on the document format, OCR techniques are first applied to extract textual information from scanned files or images.

The extracted content is subsequently cleaned and divided into semantically coherent chunks rather than fixed-length segments. Each chunk is transformed into a dense vector representation using an embedding model and indexed in the vector database. This pipeline preserves contextual integrity while enabling efficient semantic retrieval during question answering.

To maintain data isolation, documents uploaded by administrators are incorporated into the global enterprise knowledge base, whereas documents uploaded by individual users remain accessible only within their personal workspace.

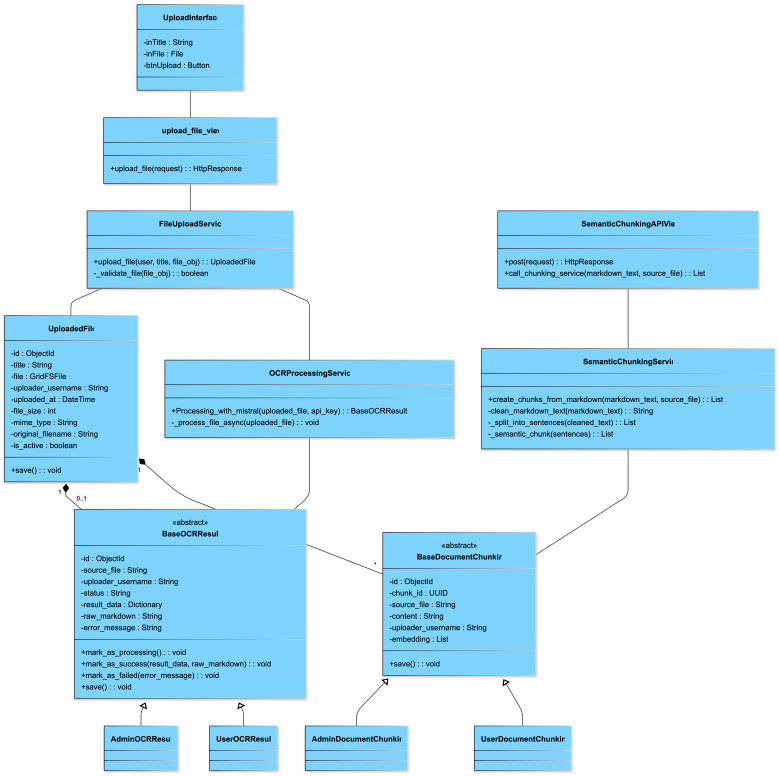


Figure 6: Class Diagram

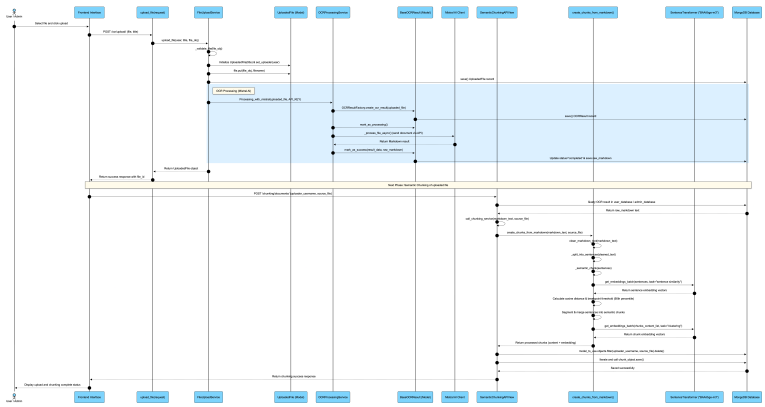


Figure 7: Sequence Diagram

4.2.4 Chatbot Interaction Module

To improve efficiency and reasoning capability, WoxionChat using **classify query agent** at the beginning of the inference pipeline. This agent analyzes the user's intent and determines whether external knowledge retrieval is necessary. If the query can be answered using the model's inherent knowledge or the available conversation memory, the system bypasses the retrieval stage

and directly generates a response. Conversely, when the query requires factual information from enterprise documents or domain-specific knowledge, the agent activates the retrieval pipeline. This selective retrieval strategy reduces unnecessary computation, lowers latency, and minimizes the inclusion of irrelevant context.
For queries requiring external knowledge, the system performs **vector retrieval**. Vector search captures semantic similarity between the query and stored documents, making it effective for paraphrased expressions and conceptual matching.

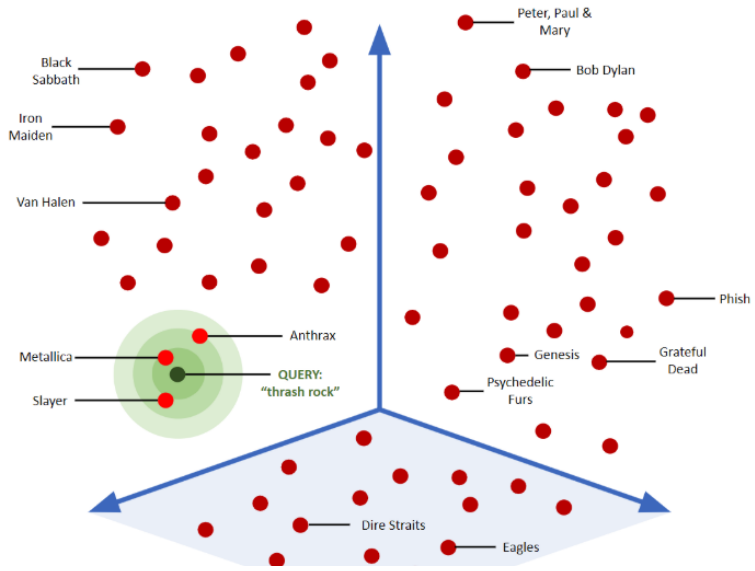


Figure 8: Vector search

To further improve evidence quality, the retrieved candidates are processed by a **reranking**. Rather than relying solely on embedding similarity scores, the reranker performs a more precise relevance assessment between the query and each candidate passage, promoting the most informative documents while filtering out redundant or weakly related results. Only the highest-ranked evidence is forwarded to the language model for response generation.
Beyond retrieval, the architecture incorporates a hierarchical memory mechanism to support long-term interactions. **Short-term memory** maintains up to the 20 most recent messages and is directly included in the inference context, enabling coherent multi-turn conversations and follow-up reasoning. To avoid excessive context growth, historical conversations are periodically compressed into **long-term memory**. If the number of recent messages exceeds 20, summarization is triggered immediately; otherwise, the system waits for approximately 30 minutes of inactivity before generating a summary. After summarization, the condensed representation is stored as persistent memory and the corresponding detailed messages are removed from short-term storage.
The proposed Agentic RAG architecture therefore consists of four tightly integrated components: (1) a classify query Agent that determines whether retrieval is necessary, (2) a vector retrieval module, (3) a reranking stage that selects the most relevant evidence, and (4) a hierarchical memory subsystem that balances recent conversational context with compressed long-term knowledge.

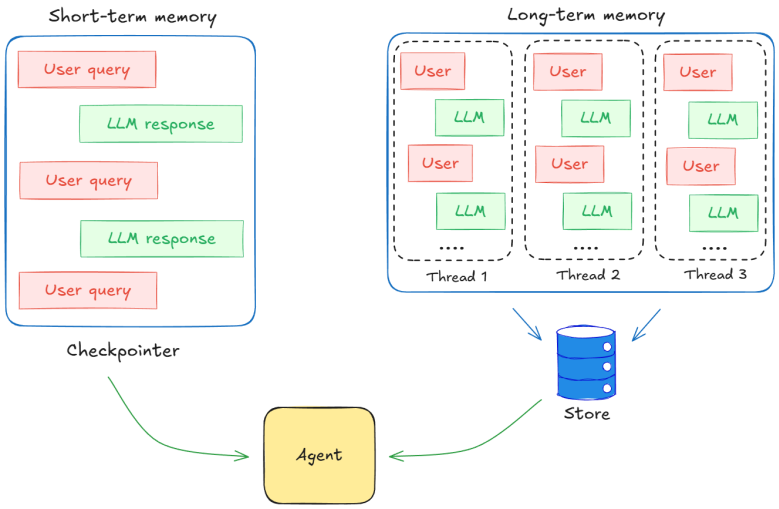


Figure 9: Type of memories

This design offers several advantages over conventional RAG systems:

- **Adaptive retrieval:** The classify query agent prevents unnecessary searches and invokes retrieval only when external knowledge is required.
- **Robust information retrieval:** Vector retrieval combines semantic understanding and lexical matching, improving coverage across diverse query types.
- **Higher evidence quality:** Reranking filters retrieved candidates and provides the LLM with more relevant supporting information.
- **Persistent conversational memory:** The combination of short-term and summarized long-term memory enables coherent interactions over extended sessions while controlling context length.
- **Efficient context engineering:** By selectively retrieving documents and compressing historical conversations, the system maximizes the informational value of the context presented to the LLM and reduces unnecessary token consumption.

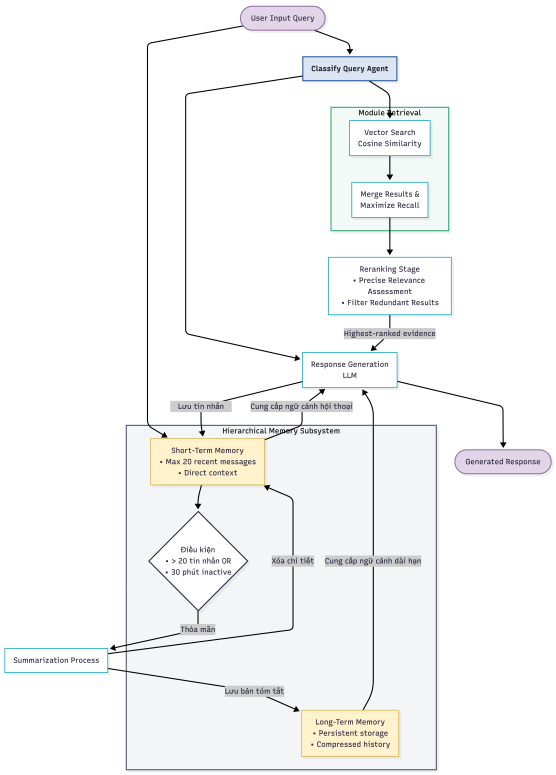


Figure 10: Agentic RAG architecture

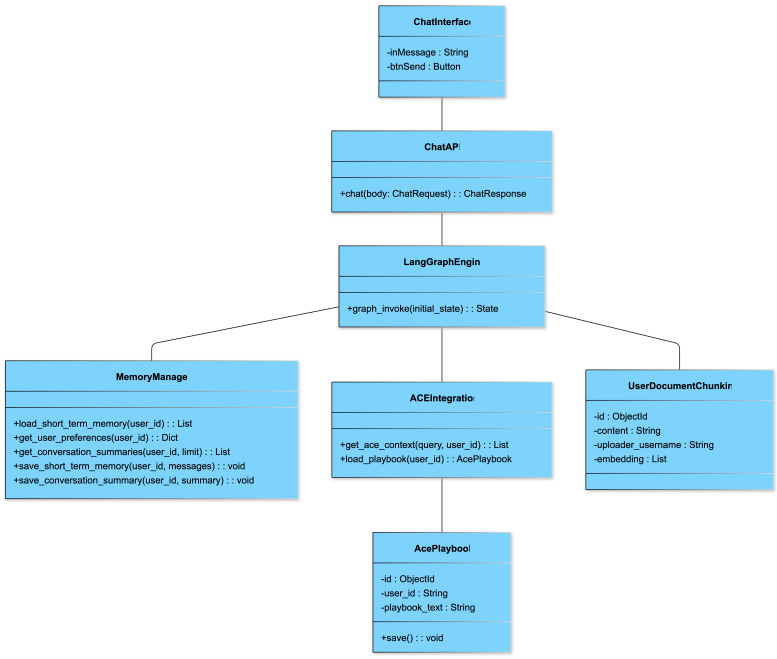


Figure 11: Class Diagram

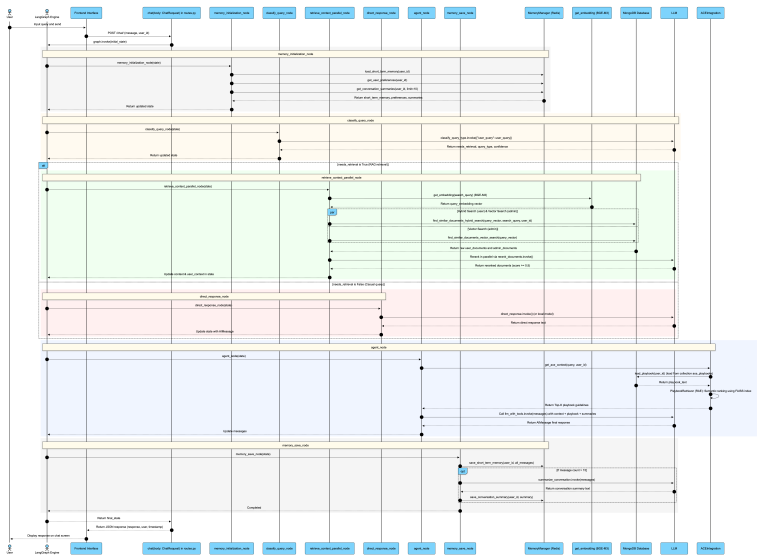


Figure 12: Sequence Diagram

4.2.5 Self-Improving Module

To enable continual improvement without retraining the underlying language model, WoxionChat integrates an enhanced Agentic Context Engineering (ACE) framework. Whenever users provide

corrective feedback, the system analyzes discrepancies between the generated response and the expected answer to identify useful insights.
The extracted knowledge is organized into structured playbooks and historical failure memories. During future interactions, instead of loading the entire accumulated knowledge base, the system retrieves only the most relevant playbook entries through Retrieval-Augmented Experience (RAE) and supplements them with semantically similar failure cases stored in the Failure Memory Bank.
This selective retrieval strategy reduces unnecessary context while allowing the language model to leverage previous successful experiences and avoid repeating known mistakes. Consequently, the chatbot continuously improves its performance over time without requiring expensive model retraining.

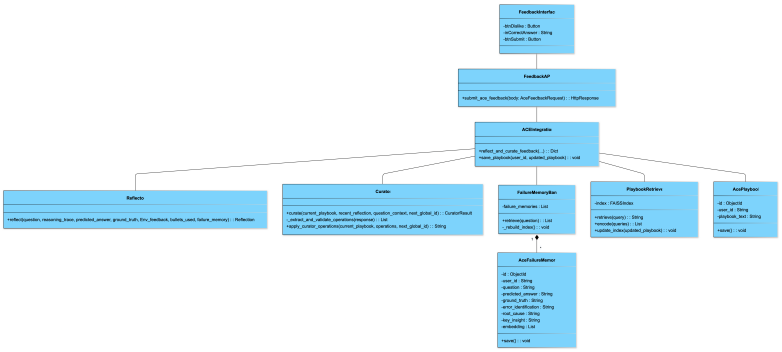


Figure 13: Class Diagram

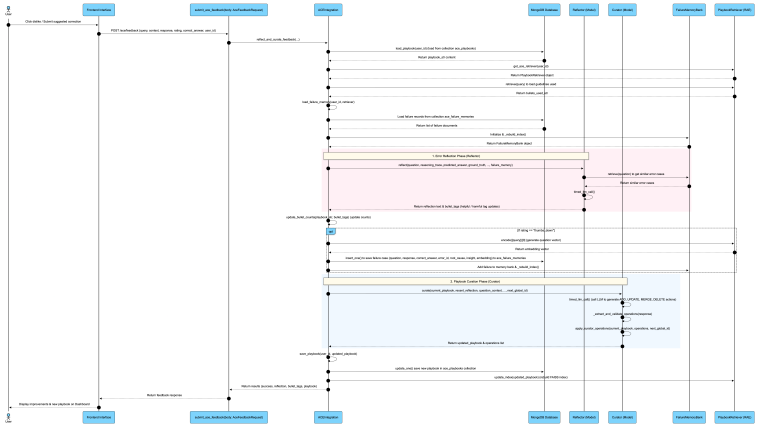


Figure 14: Sequence Diagram

4.2.6 Data design

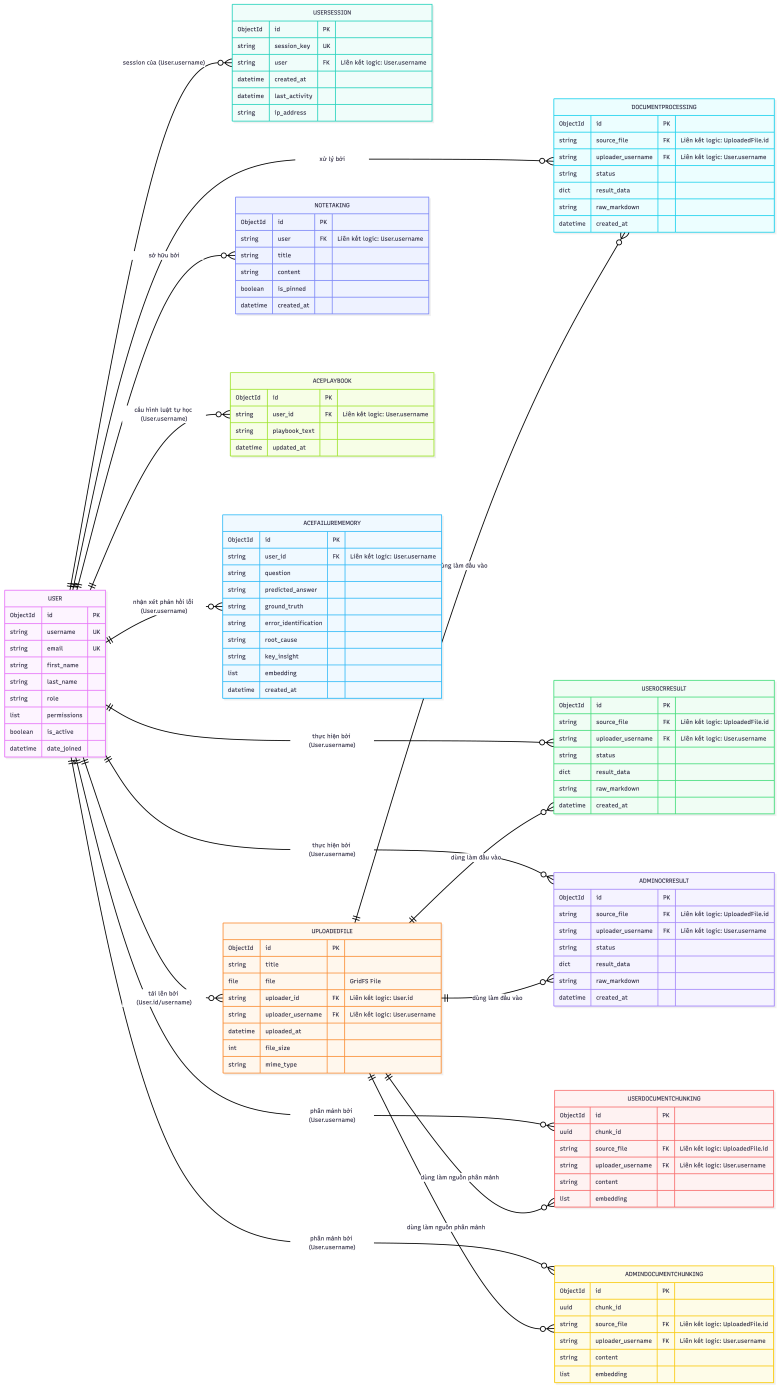


Figure 15: Data design

5 Experimental Results

5.1 Retrieval Benchmark

To evaluate the effectiveness of the proposed retrieval module, experiments were conducted on two Vietnamese retrieval benchmarks: **GreenNode/scifact-vn** and **GreenNode/scidocs-vn** in VN-MTEB [16]. Three retrieval strategies were compared, including Vector Search, Lexical Search, and Hybrid Search. The evaluation metrics include Mean Reciprocal Rank (MRR), Precision at K (P@K), and Recall at K (R@K).

Table 3: Retrieval performance on GreenNode/scifact-vn (100 queries, 1,610 documents).

Method	MRR	P@1	R@1	P@3	R@3	P@5	R@5	P@10	R@10
Vector Search (Cosine)	**0.6800**	**0.6300**	**0.5795**	**0.2467**	**0.6637**	**0.1700**	**0.7223**	**0.0930**	**0.7827**
Text Search (Lexical)	0.3160	0.2500	0.2383	0.1167	0.3258	0.0880	0.3992	0.0530	0.4775
Hybrid Search (Vector + Lexical)	0.5510	0.4800	0.4412	0.2000	0.5195	0.1480	0.6270	0.0860	0.7273

As shown in Table 3, Vector Search achieves the best performance across all evaluation metrics on the *scifact-vn* dataset, demonstrating the effectiveness of dense semantic representations for Vietnamese document retrieval. Hybrid Search provides a balance between semantic and lexical matching but remains inferior to pure vector retrieval, while Lexical Search exhibits the lowest performance due to vocabulary mismatch.

Table 4: Retrieval performance on GreenNode/scidocs-vn (100 queries, 4,431 documents).

Method	MRR	P@1	R@1	P@3	R@3	P@5	R@5	P@10	R@10
Vector Search (Cosine)	**0.3883**	**0.2900**	**0.0097**	**0.2167**	**0.0217**	**0.1620**	**0.0271**	**0.1100**	**0.0368**
Text Search (Lexical)	0.0786	0.0500	0.0017	0.0300	0.0030	0.0300	0.0050	0.0290	0.0097
Hybrid Search (Vector + Lexical)	0.3528	0.2500	0.0084	0.1600	0.0160	0.1460	0.0244	**0.1100**	**0.0368**

Table 4 shows a similar trend on the larger *scidocs-vn* benchmark. Vector Search consistently achieves the highest ranking quality as measured by MRR and P@1, while Hybrid Search remains competitive at larger retrieval depths. Lexical Search performs significantly worse across all metrics, indicating that semantic retrieval is more suitable for Vietnamese scientific document collections. These results support the adoption of embedding-based retrieval as the primary retrieval mechanism in WoxionChat, complemented by lexical retrieval and reranking within the Agentic RAG pipeline.

5.2 LLM Fine-tuning Evaluation

Table 5: Comparison of Vietnamese MMLU (VMLU) performance with existing open-source models.

Model	STEM	Social	Humanities	Others	Average
BloomVN-8B-chat	62.81	60.47	55.40	56.56	50.72
Llama3-ViettelSolution	62.42	60.12	52.37	56.20	51.52
Vintern-3B-beta	61.01	58.41	51.98	54.81	51.70
SeaLLM-7B-v2.5	60.66	55.95	49.05	53.30	49.35
MI4uLLM-7B-Chat	58.69	56.86	52.36	52.08	44.72
Vistral-7B-Chat	57.02	55.12	48.01	50.07	43.32
DeepSeek-R1-Distill-7B	46.56	39.84	38.86	48.56	61.14
SDSRV-7B-chat	60.55	55.95	49.05	48.55	36.29
Ours (Fine-tuned Qwen3-8B)	**68.15**	**63.58**	**56.37**	**56.84**	**61.95**

The results in Table 5 demonstrate that the proposed fine-tuned Qwen3-8B achieves the best overall performance among the compared Vietnamese language models, obtaining an average VMLU score of **61.95**. In particular, it reaches **68.15** on STEM, **63.58** on Social Science, **56.37** on Humanities, and **56.84** on Other domains, outperforming all competing models in the benchmark.
Compared with models of similar scale (approximately 7B - 8B parameters), including BloomVN-8B-chat, SeaLLM-7B-v2.5, MI4uLLM-7B-Chat, Vistral-7B-Chat, and SDSRV-7B-chat, the proposed model consistently achieves higher scores across most evaluation categories. These results indicate that the proposed data construction pipeline, long chain-of-thought generation strategy, and LoRA-based fine-tuning approach effectively enhance the reasoning capability and Vietnamese language understanding of the base Qwen3-8B model without increasing model size.

5.3 Enhanced Agentic Context Engineering

To ensure a fair and reproducible evaluation, all experiments were conducted under a unified experimental configuration. In particular, every compared method was executed using the same underlying language model, **Qwen3-4B-Instruct-2507**, with identical inference settings and hyperparameters. This controlled setup ensures that any observed performance differences arise from the proposed architectural modifications rather than variations in model capacity or decoding configurations.
Unless otherwise specified, all methods employ the same temperature, maximum generation length, retrieval parameters, and evaluation protocol. For approaches involving retrieval or context engineering, the retrieval backend, embedding model, and reranking strategy are kept consistent across experiments. Likewise, the same benchmark datasets and test splits are used for all evaluations.

Table 6: Financial Analysis Benchmark

Method	GT labels	Metric	Finer_0.5_offline	Formula_offline
Origin	x	Best Validation Accuracy	0.583	0.747
		Initial Test Accuracy	0.584	0.665
		Final Test Accuracy	0.513	0.700
Using RAE	x	Best Validation Accuracy	0.518	**0.787**
		Initial Test Accuracy	0.564	**0.655**
		Final Test Accuracy	0.476	**0.775**
Using Failure Memory	x	Best Validation Accuracy	**0.657**	0.740
		Initial Test Accuracy	**0.569**	0.670
		Final Test Accuracy	**0.604**	0.740
Using RAE + Using Failure Memory	x	Best Validation Accuracy	0.570	**0.787**
		Initial Test Accuracy	0.586	**0.665**
		Final Test Accuracy	0.500	**0.770**

Table 7: LLM agent Benchmark

Method	Normal_offline (%)		Challenge_offline (%)	
	Task Goal Completion	Scenario Goal Completion	Task Goal Completion	Scenario Goal Completion
Origin	20.8	8.9	6.7	0.7
Using RAE	20.2	**12.5**	8.2	0.7
Using Failure Memory	**23.2**	**12.5**	**8.9**	**2.9**
RAE + Failure Memory	21.4	10.7	7.4	2.2

6 Conclusion

This thesis presented **WoxionChat**, an enterprise-oriented AI assistant that integrates document understanding, Agentic Retrieval-Augmented Generation (Agentic RAG), large language models, and self-improving context engineering into a unified framework for intelligent enterprise knowledge management. The proposed system enables users to efficiently retrieve information from internal documents while preserving data privacy and supporting continuous adaptation through user feedback.

A key contribution of this work is the enhancement of the original Agentic Context Engineering (ACE) framework through the introduction of Retrieval-Augmented Experience (RAE) and a Failure Memory Bank, enabling more effective context selection and continual learning from historical errors. Experimental results demonstrate that these improvements significantly enhance reasoning performance, boosting the final test accuracy on complex tasks from **70.0%** to **77.0%**. In addition, the proposed fine-tuning pipeline based on carefully curated multi-domain data and long chain-of-thought supervision produces a competitive Vietnamese language model that outperforms existing models of similar parameter scale on benchmark evaluations.

Overall, the proposed architecture demonstrates that combining Agentic RAG, adaptive context engineering, and parameter-efficient fine-tuning provides an effective solution for enterprise AI assistants requiring secure knowledge access, intelligent reasoning, and long-term self-improvement.

For future work, several directions can be explored to further enhance the system. First, the retrieval pipeline can be optimized to reduce response latency by relying primarily on highly optimized **vector search**. Second, the proposed ACE framework can be further improved by developing more robust reflection and context management strategies, allowing the system to acquire higher-quality experience and prevent error propagation during continual learning. Finally, the fine-tuned language model can be enhanced through larger and more diverse reasoning datasets, improved synthetic data generation, and advanced post-training techniques, leading to stronger reasoning capabilities and better adaptation to enterprise-specific tasks.

References

[1] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Nie, and J.-R. Wen, "A survey of large language models," 2026. [Online]. Available: https://arxiv.org/abs/2303.18223

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," 2021. [Online]. Available: https://arxiv.org/abs/2005.11401

[3] Q. Zhang, C. Hu, S. Upasani, B. Ma, F. Hong, V. Kamanuru, J. Rainton, C. Wu, M. Ji, H. Li, U. Thakker, J. Zou, and K. Olukotun, "Agentic context engineering: Evolving contexts for self-improving language models," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: https://openreview.net/forum?id=eC4ygDs02R

[4] C. T. Bui, N. T. Son, T. V. Trang, L. V. Phung, P. N. Huy, H. A. Le, Q. H. Van, P. N.-T. Do, V. L. T. Truc, D. T. Chau, and L.-M. Nguyen, "VMLU benchmarks: A comprehensive benchmark toolkit for Vietnamese LLMs," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 11 495–11 515. [Online]. Available: https://aclanthology.org/2025.acl-long.563/

[5] E. J. Hu, yelong shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=nZeVKeeFYf9

[6] L. Mei, J. Yao, Y. Ge, Y. Wang, B. Bi, Y. Cai, J. Liu, M. Li, Z.-Z. Li, D. Zhang, C. Zhou, J. Mao, T. Xia, J. Guo, and S. Liu, "A survey of context engineering for large language models," 2025. [Online]. Available: https://arxiv.org/abs/2507.13334

[7] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu, "Qwen3 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[8] WHATWG, "Html living standard," https://html.spec.whatwg.org/, 2025, official HTML specification.

[9] World Wide Web Consortium (W3C), "Css: Cascading style sheets," https://www.w3.org/Style/CSS/, 2025, official CSS documentation.

[10] Django Software Foundation, "Django documentation," https://docs.djangoproject.com/, 2025, accessed: 2026-06-18.

[11] LangChain Inc., "Langchain documentation," https://docs.langchain.com/, 2025, official documentation for building LLM applications and agents.

[12] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," 2025. [Online]. Available: https://arxiv.org/abs/2402.03216

[13] E. Zacharias, M. Teuchler, and B. Bernier, "Image processing based scene-text detection and recognition with tesseract," 2020. [Online]. Available: https://arxiv.org/abs/2004.08079

[14] MongoDB Inc., "Mongodb documentation," https://www.mongodb.com/docs/, 2025, official documentation.

[15] Redis Ltd., "Redis documentation," https://redis.io/docs/, 2025, official documentation.

[16] L. Pham, T. Luu, T. Vo, M. Nguyen, and V. Hoang, "VN-MTEB: Vietnamese massive text embedding benchmark," in *Findings of the Association for Computational Linguistics: EACL 2026*, V. Demberg, K. Inui, and L. Marquez, Eds. Rabat, Morocco: Association for Computational Linguistics, Mar. 2026, pp. 1705–1725. [Online]. Available: https://aclanthology.org/2026.findings-eacl.86/